

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA0501

R.HARIKA(192572140)

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability. (BL3, BL4)*

Activity Tool : Mock Interview (Low) Assessment Tool Weightage: 30%

Dr

QUESTIONS			Total Marks:
50			
Q.No	Question	Bloom's Level	Marks
1	Interviewer: 'Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?'	BL3 – Apply	5
2	Interviewer: 'What is the difference between heuristic optimization and cost-based optimization? When would you use each?'	BL4 – Analyze	5
3	Interviewer: 'How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.'	BL3 – Apply	5
4	Interviewer: 'What are ACID properties? Give a real-world example where each property is critical.'	BL3 – Apply	5
5	Interviewer: 'Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?'	BL4 – Analyze	5
6	Interviewer: 'What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.'	BL4 – Analyze	5

7	Interviewer: 'Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?'	BL4 – Analyze	5
8	Interviewer: 'Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?'	BL3 – Apply	5
9	Interviewer: 'What is a serializability schedule? How do you test for conflict serializability using a precedence graph?'	BL4 – Analyze	5
10	Interviewer: 'Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?'	BL4 – Analyze	5

Code of Conduct:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts

Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
---------------------	----	---	--	---	-------------------------------------

I RASINENI HARIKA(192572140) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

R. Harika

Signature of the student:

RUBRICS FOR CALCULATION:

Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately

1. Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?

Answer:

Query processing converts an SQL query into an efficient execution plan and produces the result.

Steps:

1. **Parsing and translation** – SQL is checked for syntax and converted into a relational algebra/query tree.
2. **Semantic analysis** – Tables, columns, data types, and permissions are verified.
3. **Query rewriting** – Heuristic rules such as selection and projection pushdown simplify the query.
4. **Optimization** – The optimizer generates alternative execution plans.
5. **Plan selection** – It estimates the cost of each plan using factors such as disk I/O, CPU, memory, relation size, indexes, and join algorithms.
6. **Execution** – The selected plan is passed to the execution engine.

Example: For joining two large tables, the optimizer may compare nested-loop, hash, and sort-merge joins and choose the one with the lowest estimated cost.

In short:

SQL → Parse → Rewrite → Optimize → Execution Plan → Execute → Result

2. What is the difference between heuristic optimization and cost-based optimization?

When would you use each?

Answer:

Heuristic Optimization

Uses predefined rules

Does not require detailed cost calculation

Faster optimization

Example: push selections down

May not always produce the cheapest plan

Cost-Based Optimization

Uses estimated execution costs

Calculates cost of alternative plans

More computation during optimization

Example: compare different join algorithms

Usually chooses the lowest-cost plan

Heuristic rules include:

- Push selections as close to base tables as possible.
- Perform projections early.
- Execute selective operations first.
- Join smaller intermediate relations first.

Cost-based optimization uses statistics such as tuple count, number of pages, indexes, and selectivity.

When to use:

- **Heuristic:** When fast optimization is more important or the query is simple.
- **Cost-based:** For complex queries with many joins and alternative execution plans.

Conclusion: Modern DBMSs generally use cost-based optimization, often after applying heuristic transformations.

3. How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.

Answer:

In a Java project, I would use **JDBC** to connect the application to the database.

Typical implementation:

1. Add the appropriate database driver.
2. Configure the database URL, username, and password securely.
3. Create a connection.
4. Use PreparedStatement for parameterized SQL.
5. Execute the query.
6. Process the ResultSet.
7. Commit/rollback transactions where required.
8. Close resources or return connections to a connection pool.

Example:

```
Connection con = DriverManager.getConnection(url, user, password);
```

```
PreparedStatement ps =
```

```
    con.prepareStatement("SELECT * FROM Employee WHERE id = ?");
```

```
ps.setInt(1, 101);
```

```
ResultSet rs = ps.executeQuery();
```

JDBC vs ODBC:

- **JDBC:** Java-based API designed for database connectivity from Java applications.
- **ODBC:** Language-independent C-based database connectivity standard.
- JDBC uses Java interfaces and database-specific JDBC drivers.
- ODBC commonly uses ODBC drivers and is designed for broader language/platform use.

In real projects, JDBC is commonly combined with connection pooling, prepared statements, transactions, and proper exception handling.

4. What are ACID properties? Give a real-world example where each property is critical.

Answer:

ACID properties ensure reliable database transactions.

A – Atomicity:

A transaction is treated as one unit. Either all operations succeed or all are rolled back.

Example: In a bank transfer, money must be deducted from A and added to B. Both must happen, or neither should happen.

C – Consistency:

A transaction takes the database from one valid state to another valid state while maintaining constraints.

Example: A bank account should not violate a rule such as an invalid balance constraint.

I – Isolation:

Concurrent transactions should not interfere incorrectly with one another.

Example: Two customers withdrawing money simultaneously should not cause an incorrect balance.

D – Durability:

Once a transaction commits, its changes must survive system failure.

Example: After a successful money transfer, the updated balance must remain even if the server crashes.

Summary:

Atomicity → All or nothing

Consistency → Valid state

Isolation → Transactions don't interfere incorrectly

Durability → Committed data survives failure

5. Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?

Answer:

Concurrency problems occur when multiple transactions access the same data simultaneously.

1. Dirty Read:

T2 reads data written by T1 before T1 commits. If T1 rolls back, T2 has read invalid data.

2. Lost Update:

Two transactions read the same value and update it. One update overwrites the other.

3. Unrepeatable Read:

T1 reads a value twice, but T2 changes and commits the value between T1's two reads.

Two-Phase Locking (2PL) uses:

- **Shared lock (S):** For reading.
- **Exclusive lock (X):** For writing.

2PL has:

1. **Growing phase:** Transaction acquires locks.
2. **Shrinking phase:** Transaction releases locks and cannot acquire new ones.

Strict 2PL keeps exclusive locks until commit/rollback, preventing dirty reads and cascading problems.

Thus, locking ensures that conflicting operations occur in a controlled order and provides conflict serializability.

6. What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.

Answer:

A **deadlock** occurs when transactions wait for each other indefinitely.

Example:

T1 holds A and waits for B

T2 holds B and waits for A

Neither can proceed.

Deadlock Detection

A **Wait-For Graph (WFG)** is constructed:

- Each transaction is represented by a node.
- Edge T1 → T2 means T1 is waiting for T2.
- A cycle indicates a deadlock.

Example:

T1 → T2 → T3 → T1

The system detects the cycle and aborts/rolls back one transaction.

Prevention Strategies

1. Wait-Die:

- Older transaction can wait for younger transaction.
- Younger transaction requesting an older transaction's lock is aborted.

2. Wound-Wait:

- Older transaction requesting a younger transaction's lock aborts the younger transaction.
- Younger transaction waits for older transaction.

Other methods include **timeout-based detection** and ordering resources to prevent circular waits.

7. Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?

Answer:

Immediate Update

Database may be updated before commit

Requires UNDO and REDO

Uses log records with old/new values

More flexible

Recovery is more complex

Deferred Update

Database is updated after commit

Generally requires REDO only

Updates of uncommitted transactions aren't written to database

Simpler recovery

Recovery is relatively simple

Immediate Update:

Committed → REDO

Uncommitted → UNDO

It is preferred when the DBMS needs to allow updated pages to be written before transaction commit.

Deferred Update:

Committed → REDO

Uncommitted → Ignore

It is preferred when simpler recovery is desired and uncommitted changes can be kept out of the database until commit.

Conclusion: Immediate update provides greater flexibility, while deferred update simplifies recovery.

8. Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?

Answer:

Shadow Paging is a recovery technique that maintains two page tables:

- **Shadow page table:** Points to the stable database pages.

- **Current page table:** Points to pages being modified.

When a page is modified, a new copy is created instead of overwriting the original page.

Shadow Table → Original stable pages

Current Table → Modified pages

On commit: The current page table becomes the new stable page table.

On crash: The system discards the current page table and uses the shadow page table.

Advantages

- Simple recovery mechanism.
- No UNDO/REDO logging required.
- Fast rollback.
- Original pages remain safe until commit.

Limitations

- Maintaining page tables can require additional overhead.
- Fragmentation may occur.
- Copying and managing pages can be expensive.
- Less suitable for very large databases and high-concurrency systems.

Compared with log-based recovery, shadow paging is simpler, but modern large DBMSs commonly prefer sophisticated log-based techniques such as ARIES.

9. What is a serializability schedule? How do you test for conflict serializability using a precedence graph?

Answer:

A **serial schedule** executes transactions one after another without interleaving.

A **serializable schedule** may interleave transactions but produces the same result as some serial execution.

For **conflict serializability**, construct a **precedence graph**.

Steps:

1. Create one node for each transaction.
2. Identify conflicting operations:
 - Read–Write
 - Write–Read
 - Write–Write
3. If an operation of T_i occurs before a conflicting operation of T_j , add:

$T_i \rightarrow T_j$

4. Check the graph for cycles.

Example:

$T_1: W(A)$

$T_2: R(A)$

Since T_1 writes A before T_2 reads A :

$T_1 \rightarrow T_2$

If the complete graph is:

$T_1 \rightarrow T_2 \rightarrow T_3$

with **no cycle**, the schedule is conflict-serializable.

If:

$T1 \rightarrow T2 \rightarrow T1$

a cycle exists, so the schedule is **not conflict-serializable**.

10. Describe Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?

Answer:

1. Basic 2PL

Has two phases:

- **Growing:** Acquire locks.
- **Shrinking:** Release locks.

Once a transaction releases a lock, it cannot acquire another lock.

Guarantee: Conflict serializability.

Problem: Deadlocks can occur.

2. Strict 2PL

All **exclusive (X) locks** are held until the transaction commits or aborts.

Advantages:

- Prevents dirty reads of uncommitted writes.
- Prevents cascading rollbacks.
- Simplifies recovery.

3. Rigorous 2PL

Both **shared (S) and exclusive (X) locks** are held until commit or abort.

It is stricter than Strict 2PL.

Protocol	Lock Release
-----------------	---------------------

Basic 2PL	During shrinking phase
-----------	------------------------

Strict 2PL	X-locks held until commit
------------	---------------------------

Rigorous 2PL	S-locks and X-locks held until commit
--------------	---------------------------------------

Most commonly used: Strict 2PL is widely used because it provides serializability while simplifying recovery and preventing cascading rollbacks, without being as restrictive as rigorous 2PL.